DUMBATHON 2.0

// APOCALYPSE PROTOCOL //

APP / WEB DEVELOPMENT TRACK

SPECIFICATION REQUIREMENTS DOCUMENT

THE GRID IS DOWN. SERVERS ARE BURNING. THE OLD RULES ARE GONE. BUT YOUR TEAM HAS ONE JOB: BUILD. THESE SPECIFICATIONS ARE YOUR FIELD MANUAL. DEVIATE FROM THEM AND YOUR PROJECT IS DEAD ON ARRIVAL. FOLLOW THEM — AND MAYBE, JUST MAYBE, YOU SURVIVE THE JUDGING.

APPROVED

//1 - FUNCTIONAL REQUIREMENTS:

[1.1 CORE APPLICATION FUNCTIONALITY]

- Each application must successfully implement its primary purpose:
- Alarm clock: Alarm must be set, triggered, and stopped through the required interaction mechanism
- Map Application: Map must display current location and provide route navigation.
- E-commerce: Users must be able to browse items and successfully complete a barter/add-to-cart interaction
- Journal: Users must be able to create, edit, and save journal entries
- The core feature must remain usable even after implementing all dumb challenges.

[1.2 GLOBAL DUMB CHALLENGE IMPLEMENTATION]

Every application must implement all three global dumb challenges:

1) FACE RECOGNITION ACCESS CONTROL:

- The application must only open if Fariz’s face is detected.
- If the face is not detected or does not match, the app must display a lock screen with a denial message.
- The system must use real-time camera input.

2) RAGEBAIT CAPTCHA:

- A custom CAPTCHA must appear at app entry (after face unlock), designed to be intentionally frustrating.
- Examples: blurry distorted images, trick questions, moving targets, impossible grids, misleading instructions.
- The CAPTCHA must still be passable — it should be hard and annoying, not literally unsolvable.
- Flavor it apocalypse-style: 'Identify all safe zones', 'Select all images that are NOT on fire', etc.

3) RANDOM POPUP INTERRUPTIONS:

- Every time the user attempts a primary action (submit, confirm, navigate, purchase, save), a random popup must appear first.
- The popup must interrupt the action — the action should NOT complete until the popup is dismissed.
- Popup content must be randomized each time — same popup every time is not acceptable.
- Popup themes must be apocalypse-flavored: fake system warnings, faction alerts, supply shortages, cryptic messages.

[1.3 APP-SPECIFIC DUMB CHALLENGE IMPLEMENTATION]

Each app must also implement its assigned dumb challenge.

1) ALARM CLOCK:

- The alarm cannot be dismissed by tapping a simple stop button
- The user must complete a specific physical or cognitive challenge to stop it — e.g. solve a math problem, shake the device in a pattern, type a passphrase.
- The challenge must be randomly selected from a pool each time the alarm fires — not the same task every time.
- If the challenge is failed or ignored, the alarm must escalate (louder, more frequent, flashing UI).

2) MAP APPLICATION:

- By default, the map must calculate and display a route that is 10x longer than the actual optimal path.
- The user must tap a rhythmic pattern to calm the GPS and restore normal routing.
- Paranoid mode must re-activate after a set time or on route recalculation — it cannot be permanently disabled.(??)

3) E-COMMERCE(BARTER SYSTEM):

- The Add to Cart button runs away from the cursor.
- Movement must be reactive — the button detects cursor proximity and flees in the opposite direction.
- The button must have a visible panicked state — shaking animation, distress icon, frantic movement speed.(??)

4) JOURNAL / BLOGGING (STILL ALIVE JOURNAL – SURVIVOR'S LOG):

- The journal must automatically delete whatever the user typed in the last 60 seconds unless the user taps a "Still Alive" button.
- The "Still Alive" button must appear randomly somewhere on the screen every minute, forcing the user to confirm they are still present.
- If the user does not tap the button in time, the last 60 seconds of typed content must be erased automatically.
- A visible indicator must show that the system is waiting for the "Still Alive" confirmation (e.g., countdown timer, warning icon, or alert message).
- Teams must implement a smart recovery or auto-save mechanism that silently backs up the user's text periodically so deleted content can be restored if the user misses the confirmation.

//3 - TECHNICAL REQUIREMENTS:

[3.1 Supported Platforms]

Teams may develop the application for:

- Web applications (preferred)
- Android applications
- iOS applications
- Web applications must run in modern browsers (Chrome/Edge/Firefox).